TP2 Main Video Scripts, Team 24

Course: CSE 360, 2026 Fall C — Team 24: Arush Ugal, Karan Patel, Anirudh B, Pranjal Ganvir, Moksha Dev

This document covers the three main TP2 screencasts. The six standup summaries are in `TP2 STANDUP SCRIPTS.md`.

Your TA confirmed you may record all videos together in one session, and that an end-to-end walkthrough is fine as long as the whole submission is present and every rubric item is covered.

The three screencast deliverables

File to produce	What it is	Duration	Covered here
LessonsLearnedScreencast.mp4	Explains the Task 3 Javadoc for both entity classes	5-10 min	Part 1
PhaseTwoGUIScreencast.mp4	Explains the Task 4 Javadoc for the GUI MVC packages	5-10 min	Part 2
TestingScreencast.mp4	Explains Task 7 test Javadoc AND shows each test executing	8-15 min	Part 3

Pairings (each screencast is a two-person collaboration, every member presents, time split evenly): - LessonsLearnedScreencast: **Arush + Anirudh** - PhaseTwoGUIScreencast: **Moksha + Karan** - TestingScreencast: **Pranjal + Anirudh**

Setup for all three

1. Join one Zoom meeting, cameras off, record to the **Cloud**.
2. Share the **whole screen** so the viewer sees you move between the browser (Javadoc HTML), Eclipse (source), and the running app.
3. Eclipse text font 16+. Small text loses marks directly (rubric 9.0).
4. Do Not Disturb on; close all notifiers.
5. Each file must be MOV or MP4 and under 100 MB, or the deliverable scores zero.
6. Confirm durations: Parts 1 and 2 are 5-10 min; Part 3 is 8-15 min.

Part 1. LessonsLearnedScreencast (5-10 min). Arush + Anirudh

Goal: show and explain the contents of `Lessons_Learned_Javadocs` for both classes, show the list of deliverable requirements, show source/comment screenshots satisfying each, and show the generated Javadoc pages. (Rubric 9.1)

Speaker 1. Arush. The LessonsLearned class

Open first: the generated Javadoc page `Lessons_Learned_Javadocs/entityClasses/LessonsLearned.html` in the browser, and `entityClasses/LessonsLearned.java` in Eclipse beside it.

Say: This is Team 24's LessonsLearned screencast. I am covering the `LessonsLearned` entity class. It lives in the `entityClasses` package, next to the `User` class that came with Foundations, so a maintainer finds it exactly where they expect. Point at deliverable 3.1 screenshot showing the package.

Walk the class purpose from the Javadoc: this class holds all the data for one lesson learned and provides the full CRUD set — create, read, update, delete — plus input validation on every field, as required by the Contributor user stories.

Point at each attribute in the Javadoc and give its source and rationale, because the rubric (3.3, 3.4) specifically wants the source and rationale for each attribute, not just its name. For each one say: this attribute, this is the Contributor story it comes from, and this is why it is typed the way it is. Mention which attributes exist to support the TP3 epics, so the class will not need reworking later — that is an explicit requirement of Task 3.

Then switch to the source and show the create and read methods with their internal comments. Point at a comment that explains which line satisfies which requirement (deliverable 3.5).

Hand over to Anirudh.

Speaker 2. Anirudh. The LessonsLearnedList class

Open first: `Lessons_Learned_Javadocs/entityClasses/LessonsLearnedList.html` and `entityClasses/LessonsLearnedList.java`.

Say: I am covering `LessonsLearnedList`. It supports the CRUD actions for lists of lessons. One instance is the persistent list of every lesson in the system; others are subsets, such as the list of lessons matching a search keyword, which is what the Contributor search story needs.

From the Javadoc, walk the operations: add, remove, retrieve, the all-lessons persistent list, and the keyword-search subset. For each, point at the source and rationale the way Arush did.

Make the testability point, the same one we made in TP1: these classes contain no JavaFX, on purpose, so the same code the screens call can be driven directly

by the tests. That is why the documents, the code, and the tests describe one system.

Close: that is the Lessons Learned documentation. Both classes have full CRUD, validated input, every attribute traced to a requirement, and Javadoc written as if by the original Foundations authors.

Part 2. PhaseTwoGUIScreencast (5-10 min). Moksha + Karan

Goal: show and explain `Contributor_Javadocs` — the new GUI MVC packages, that they follow the Foundations pattern, and that they cover all Contributor stories. (Rubric 9 / Task 4)

Speaker 1. Moksha. The Create/Edit GUI package

Open first: the Javadoc for the lesson-create GUI package and the three MVC source files (`ModelLessonCreate`, `ViewLessonCreate`, `ControllerLessonCreate`) in Eclipse.

Say: I am covering the GUI that lets a contributor create and edit a lesson. It is a new package named consistently with the supplied Foundations GUI packages like `guiNewAccount`, and it is split into Model, View, and Controller exactly like them, so maintainers find the code where they expect.

Point at the view building the controls, then the controller. Stress the layering: the controller validates every field through the `LessonsLearned` entity class and never touches the database directly, exactly as the Foundations controllers do. Show where the coaching hooks from AICoaching attach to the description and title fields, while the mandatory length and format checks stay in place.

From the Javadoc, read the purpose, the attributes, and the source/rationale, as the rubric requires (4.4). Hand over to Karan.

Speaker 2. Karan. The List/Search GUI package

Open first: the Javadoc for the lesson-list GUI package and its MVC source files.

Say: I am covering the GUI that lists lessons and searches them. Same MVC shape, same naming convention. The controller calls `LessonsLearnedList` for its data, including the keyword-search subset, and renders the results.

Walk the Javadoc for this package: purpose, classes, attributes with source and rationale, and the operations. Point at an internal comment that explains a non-obvious line (deliverable 4.5).

Close: between the two packages, every Contributor user story has a window, every window follows the Foundations MVC pattern, and the Javadoc documents how each one satisfies its story.

Part 3. TestingScreencast (8-15 min). Pranjal + Anirudh

Goal: explain the Task 7 test Javadoc AND show each test actually executing, explaining why the output convinces the viewer the test and the code-under-test both work. (Rubric 9.1 for Task 7)

Speaker 1. Pranjal. The test design and the Javadoc

Open first: `ContributorTests_Javadocs` in the browser and the `testingAutomation` package in Eclipse, plus the Requirements to Tests Map PDF.

Say: I am covering the Contributor tests. There is one suite per CRUD area plus a `TP2TestRunner` that runs them all and totals the result, the same shape as the `TP1TestRunner` we wrote last phase. Each test supplies one input, states whether it should be accepted, calls the same code the screens call, and counts a pass or a fail.

Open the Requirements to Tests Map and show that every Contributor requirement has at least two tests — a positive and a negative — and names the Javadoc HTML file that documents each (Task 7.2).

Open one test's Javadoc page and show it names which requirement is tested, how the code satisfies it, and how to read the output to know it passed (Task 7.3). Point at the source screenshot that captures that test with comments (Task 7.4).

Hand over to Anirudh to run them.

Speaker 2. Anirudh. Executing the tests

Open first: Eclipse, `testingAutomation/TP2TestRunner.java`.

Say: now I run them. Right-click `TP2TestRunner`, Run As, Java Application. Let it finish and scroll to the bottom of the console.

Read the result aloud: X passed, Y failed. If any supplied expected-fail cases were kept, explain them immediately the way we did in TP1 — they assert the wrong answer on purpose, the runner knows about them, and it prints that every test behaved as expected.

Then pick one requirement and show both its tests running: the positive case being accepted and the negative case being correctly rejected. Explain why the pair together proves the requirement — the positive shows the feature works, the negative shows it refuses what it should refuse, and the boundary is in the right place.

Close: every Contributor requirement has a passing positive test and a correctly-failing negative test, each documented in the Javadoc and captured in a screenshot. That is the end of our testing screencast. Thank you for watching.

Optional: end-to-end demo segment (TA-approved)

The TA said an end-to-end situation is fine. If you want one continuous walk-through to tie it together, record it as an extra segment at the end of the TestingScreencast (keeping total under 15 min) or as its own clip in the Planning folder — but do NOT let it replace the three named screencasts above, since the rubric grades those by name. Suggested flow: create a lesson in the GUI (bad input refused, then good input accepted and coached), see it appear in the list, search for it by keyword, edit it, delete it, then show the test runner going green. One person narrates, the other drives.